LAB University of Applied Sciences

VI - Log Analysis and Logging in DevOps

LAB University of Applied Sciences
Bachelor of Engineering, Industrial Information Technology 24S
AT00CK37-3004 DevOps Engineering - 2026
Parth Mangukiya

Abstract

Author(s)	Type of publication	Published in (year)
Parth Mangukiya	Course Report	2026
	Number of pages 14	

Title of the publication

Log Analysis and Logging in DevOps

Degree, Field of Study

Bachelor of Engineering, Industrial Information Technology

Commissioning organization (if the thesis is commissioned by another party)

Abstract

This report investigates an application failure by analysing application and system log files. Microsoft 365 Copilot was used to support the initial analysis and identify the likely sequence of events. The findings were then checked against the original log entries. The analysis indicates that high memory usage led the operating system to terminate the MySQL process, after which the database service failed and the application lost its database connection. The report also discusses the limitations of the available log data, the importance of clear log messages, the use of logging levels and the possible disadvantages of excessive DEBUG logging. The work highlights the need to verify AI-generated conclusions rather than treating them as confirmed facts.

Keywords

DevOps, log analysis, troubleshooting, logging levels, artificial intelligence

Contents

1. Introduction 1
2. Task 1 – AI-Assisted Log Analysis 1
2.1 AI Tool and Model 1
2.2 Summary of AI Conclusions 1
2.3 AI Prompt and Response 2
3. Task 2 – Verification of AI Conclusions 2
3.1 Comparing the AI analysis with my own findings 2
3.2 What the AI could not establish 2
3.3 Additional observations from checking the logs 3
3.4 Self reflection and AI Findings 4
3.5 Summary of Task 2 4
4. Task 3 – Opinions on Logging 5
4.1 Why are logs essential in DevOps? 5
4.2 What makes a good log message? 5
4.3 When would DEBUG logs be harmful? 5
4.4 Are logging levels used consistently? 6
4.5 Summary and Personal Reflection 6
5. References 7
Appendix 8
Appendix A – AI Prompt 8
Appendix B – Microsoft Copilot Response 9

Appendix A – AI Prompt

Appendix B – AI Response

1. Introduction

Logging is an important part of DevOps because it helps developers and system administrators monitor applications, identify errors and investigate system failures. Application and operating-system logs provide information about events that occur during the operation of a service. By comparing different log sources, engineers can understand the sequence of events and identify possible causes of problems.

The objective of this assignment is to investigate a malfunctioning service using two provided files: application.og (application log) and system.log (system log). The analysis focuses on understanding what happened, identifying the root cause and examining the use of different logging levels.

ChatGPT was used to support the analysis of the log files. Its conclusions were then compared with the original log entries. The final part of the assignment discusses the importance of logging in DevOps and the advantages and disadvantages of different logging levels.

2. Task 1 – AI-Assisted Log Analysis

2.1 AI Tool and Model

Microsoft Copilot was used to analyse the application and system log files. The model was identified in the Copilot conversation as GPT-5 chat model. The purpose of using the AI was to understand the events recorded in the logs, identify the most likely reason for the service failure and explain the logging levels.

The prompt and the response received from Copilot are included in Appendix A and Appendix B.

2.2 Summary of AI Conclusions

Copilot's analysis showed that the application initially started normally and established a connection to the database. The first request, GET /users, was processed successfully. During the next request, GET /orders, the application started experiencing database problems. The query failed, the retry timed out and the application returned an HTTP 500 error.

The system log provides the important information about the database failure. It records memory usage reaching 92%, followed by the operating system terminating the mysqld process through an out-of-memory (OOM) event. The database service then failed to restart.

As a result, the application could not establish a new database connection. It entered degraded mode and eventually shut down. Copilot identified memory exhaustion as the most likely immediate cause of the incident. However, the available logs do not establish what originally caused the high memory usage.

2.3 AI Prompt and Response

The complete AI prompt and response are included in the appendices. Appendix A contains the exact prompt used during analysis and Appendix B contains the full AI response.

3. Task 2 – Verification of AI Conclusions

3.1 Comparing the AI analysis with my own findings

After reading the application log first, I understood that the application had lost its connection to the database. At that point, I was not sure why the database had stopped working. I then examined the system log and found the memory warning and the message showing that the operating system had killed the mysqld process.

This helped me understand the connection between the two files. The application log shows the problems experienced by the application, while the system log gives evidence of what happened to the database service. I agree with Copilot's main conclusion that the out-of-memory (OOM) event and the termination of the database process were closely related to the application failure.

The AI analysis was useful because it brought together events from both files and explained how one problem led to another. However, I did not accept the conclusion only because the AI suggested it. I checked the relevant messages myself, especially the high memory warning, the OOM kill message and the database service failure. These entries support the explanation provided by Copilot.

3.2 What the AI could not establish

Although the logs show that the OOM killer terminated mysqld, they do not identify the process that originally caused the high memory usage. It would therefore be incorrect to say that MySQL itself was responsible for consuming all the memory. The logs only show which process was terminated.

I also noticed two different memory measurements. The system log reports memory usage of 92%, while the application log records current_memory=512MB with a threshold of

500MB. These values come from different log sources and are expressed differently. The available information does not explain whether they measure the same memory resource, so I cannot directly compare them.

Another limitation concerns the database restart. The system log says that the restart failed and that memory was still critical. However, it does not provide the exact reason why the restart attempt failed. More information, such as system metrics, database service diagnostics or monitoring data, would be needed to investigate that part.

This shows that AI can identify a likely explanation from the available evidence, but it cannot provide information that is not recorded in the logs. In this case, the original source of the memory pressure remains unknown.

3.3 Additional observations from checking the logs

One message that caught my attention was: WARNING - Database might be slow

This appears after the application has already recorded a failed reconnection with the message Reconnect failed: connection refused. In that situation, describing the database as possibly slow does not accurately communicate the problem. The database appears to be unreachable, rather than simply responding slowly.

I also checked the final application message: ERROR - Critical failure: shutting down application

The severity recorded at the beginning of this entry is ERROR. The word "Critical" describes the failure in the message itself; it does not make the entry a CRITICAL-level log. This distinction is important when discussing the logging levels.

The application log records the events with millisecond timestamps, while the system log shows timestamps only to the nearest second. The times are useful for building a general sequence of events, but they do not prove the exact order of events that occurred within the same second.

These details demonstrate why it is important to read the original log messages carefully instead of relying only on an AI-generated summary. Some messages may be vague, and the severity level and the wording of a message do not always mean the same thing.

3.4 Self reflection and AI Findings

At the beginning of the task, I mainly focused on the application log. I could see that the database connection was lost and that the application returned an HTTP 500 error, but I did not immediately understand why the database had stopped responding. My first impression was that the problem was somewhere in the application or its database connection.

After examining the system log, I found the message showing that memory usage had reached 92% and that the operating system had killed the mysqld process. This was an important point in my investigation because it helped me understand that the application failure was connected to a problem at the system level.

Copilot helped me connect the events from the two files and present them in a clear sequence. It explained that the database process was terminated because of memory pressure, the application could no longer communicate with the database, and the service eventually shut down. This made the overall incident easier for me to understand.

However, my own review also helped me notice details that needed more careful interpretation. For example, the logs do not reveal which process originally caused the high memory usage. They also do not explain exactly why the database restart failed. I therefore learned that an AI explanation should be treated as a starting point for investigation rather than as a final answer.

The most useful part of this exercise was comparing the application and system logs instead of studying each file separately. I learned that an application error may be only the visible result of a problem elsewhere in the system. Looking at related logs can provide a much better understanding of the complete incident.

3.5 Summary of Task 2

After checking Copilot's conclusions against the original log files, I agree with its main finding: memory pressure led to the operating system terminating the database process, which caused the application to lose its database connection and eventually shut down.

My manual investigation supports this conclusion through the OOM message, the failed database restart, the application reconnection error and the final shutdown message. I also identified limitations that should not be overlooked. The logs do not show what originally

caused the memory pressure, and they do not provide the exact reason for the failed database restart.

Overall, this task showed me that AI can make log analysis faster and help identify relationships between events. However, manually checking the evidence is still necessary to avoid accepting assumptions as facts. Combining AI assistance with my own analysis gave me a more complete understanding of the incident.

4. Task 3 – Opinions on Logging

4.1 Why are logs essential in DevOps?

In my opinion, logs are essential because they help engineers understand what a system was doing before and during a failure. They are especially useful when a problem cannot easily be reproduced. In this exercise, the application log showed the database connection errors, but the system log helped explain why the database process stopped. Without both sources, it would have been more difficult to understand the incident. NIST also describes log management as an important part of identifying and investigating operational and security incidents (Kent and Souppaya, 2006).

4.2 What makes a good log message?

A good log message should make it possible to understand what happened without having to guess. It should include a useful timestamp, the affected component, the event and enough technical detail to support further investigation. For example, Out of memory: Kill process 2451 (mysqld) identifies the affected process and the action taken by the operating system.

A less useful example is Database might be slow. It does not include a response time or other measurement, and it appears after a connection refusal has already been recorded. In my view, a message such as Database connection unavailable after reconnect attempt would describe the situation more accurately. OWASP recommends recording enough event information to support later monitoring and analysis (OWASP Foundation, n.d.).

4.3 When would DEBUG logs be harmful?

DEBUG messages are useful during development and troubleshooting, but leaving very detailed logging enabled all the time can create problems in production. A large number of messages can increase storage requirements and make important errors harder to find. They can also expose internal technical information. For example, this application log includes db_host=localhost and an SQL query. These entries are useful for debugging, but developers

should consider whether such details are necessary in production and restrict access to the logs. The Python Logging HOWTO describes DEBUG as detailed information mainly intended for diagnosing problems (Rantanen 2026; Python Software Foundation, n.d.).

4.4 Are logging levels used consistently?

The application log generally distinguishes normal events from problems. Startup and successful requests are recorded as INFO, while database problems and failed operations are recorded as ERROR. Warnings are also used for unusual conditions, such as high memory usage.

However, I think the entry Health check failed: database unreachable should be reviewed. It is recorded as DEBUG even though the database is an important dependency and the application has already entered degraded mode. Depending on the purpose of the health check, WARNING or ERROR might communicate the seriousness more clearly.

The system log uses a different format, identifying sources such as kernel and systemd rather than displaying the same explicit severity labels as the application log. This makes it harder to compare the severity levels directly between the two files. Overall, the logs contain useful information, but clearer and more consistent formatting would make the investigation easier.

4.5 Summary and Personal Reflection

This task helped me understand that logging is an important part of maintaining reliable applications and services in a DevOps environment. Logs are not only useful when something fails; they also help developers and system administrators understand normal system behaviour, monitor performance and identify possible problems before they become serious incidents.

I learned that good log messages should be clear, specific and useful for troubleshooting. A message should explain what happened and provide enough information to help someone investigate the issue. For example, a message such as Database might be slow is not very helpful because it does not explain the actual problem. A more useful message would identify that the database connection was refused and include relevant details, such as the time, service and error information.

I also understand better why choosing the correct logging level matters. DEBUG messages are useful during development and detailed troubleshooting, but too many DEBUG

messages in a production environment can make important events harder to find and increase storage requirements. INFO messages should describe normal operations, WARNING messages should highlight possible problems, and ERROR messages should indicate failures that need attention. These levels should be used consistently so that people can understand the seriousness of an event.

My main learning from this exercise is that logging should be planned as part of application development, rather than treated as something to add only after a problem occurs. Well-written and consistent logs make troubleshooting easier and help teams understand incidents more quickly. In a DevOps environment, this supports monitoring, communication and continuous improvement.

Use of Artificial Intelligence

The author, Parth Mangukiya, used Microsoft 365 Copilot (GPT-5 chat model) to support the analysis of the application and system log files. The tool was used to identify the sequence of events, suggest the likely cause of the service failure and explain the logging levels. The AI-generated analysis was checked against the original log entries, and the conclusions were reviewed before being included in this report. The author takes full responsibility for the content of the report and has critically evaluated the information produced by the AI tool.

5. References

Kent, K. and Souppaya, M. (2006). *Guide to Computer Security Log Management*. NIST Special Publication 800-92. National Institute of Standards and Technology. Available at: https://doi.org/10.6028/NIST.SP.800-92 (Accessed: 11 November 2026).

OWASP Foundation (n.d.). *Logging Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html (Accessed: 11 November 2026).

Python Software Foundation (n.d.). *Logging HOWTO*. Python Documentation. Available at: https://docs.python.org/3/howto/logging.html (Accessed: 11 November 2026).

Rantanen, P. 2026. Operate and Monitor. AT00CK37-3004 DevOps Engineering (Software Testing), Week 45 lecture slides, 5 November 2026. LAB University of Applied Sciences.

Appendix

Appendix A – AI Prompt

I am completing a DevOps log analysis assignment. Please analyze the following log files (application.log and system.log) and answer:

Please analyze the following log files (application.log and system.log) and answer:

1. Explain what is happening in the logs.
2. Identify the most likely root cause of the failure.
3. Explain the meaning of the log levels used (DEBUG, INFO, WARN, ERROR, etc.).
4. Provide a short summary of your conclusions.

Application log:
[PASTE application.log HERE]

System log:
[PASTE system.log HERE]

Appendix B – Microsoft Copilot Response

Perfect! Now you have both logs, so you can complete Task 1 properly. Here's a ready-to-use analysis.

Task 1 – AI Analysis

AI Tool Used

Microsoft 365 Copilot (GPT-5 chat model)

AI Analysis

1. What is happening in the logs?

The application starts successfully and loads its configuration. A database connection is established, and the service begins accepting requests.

The first request (GET /users) is processed successfully. However, during the second request (GET /orders), the application reports that the database is responding slowly and then loses the database connection.

The application retries the query, but the retry times out. As a result, a 500 Internal Server Error is returned to the client.

The application then attempts to reconnect to the database, but the reconnect fails because the connection is refused. The application enters degraded mode, health checks fail, and eventually the application performs a shutdown due to a critical failure.

Meanwhile, the system log shows that memory usage rises to 92%, causing the operating system's Out-Of-Memory (OOM) killer to terminate the database process (mysqld). Afterward, systemd attempts to restart the database service but fails. Since the database never comes back online, the application cannot reconnect and eventually shuts down.

2. Most Likely Root Cause

The root cause is memory exhaustion on the server.

Evidence:

system.log

- Memory usage high: 92%
- Out of memory: Kill process 2451 (mysqld)

The operating system terminated the database process because memory usage became critical.

After the database was killed:

- Database Service entered failed state

- Failed to start Database Service

The application symptoms in application.log are consequences of the database failure:

- Query failed: database connection lost
- Reconnect failed: connection refused
- Health check failed: database unreachable
- Critical failure: shutting down application

Therefore:

Root Cause → Server runs out of memory → MySQL process is killed → Database becomes unavailable → Application fails and shuts down.

3. Explanation of Log Levels

DEBUG: Detailed technical information used for troubleshooting and development.

INFO: Normal operational events showing expected behavior.

WARNING: Potential problem or abnormal condition that does not immediately stop execution.

ERROR: Serious issue causing a function or operation to fail.

CRITICAL*: Very severe error causing service shutdown or major outage.

Although "CRITICAL" is not one of the commonly required log levels, it is often used for fatal system failures.

4. Short Summary

The application starts normally and initially works correctly. A sudden increase in memory usage causes the operating system to kill the MySQL database process. The application's database requests then fail, reconnect attempts are unsuccessful, and health checks report the database as unreachable. Because the database service cannot be restarted, the application enters degraded mode and eventually shuts down completely.

Example Task 2 Reflection

Do you agree with the AI's conclusions?

Yes. The evidence from both logs clearly shows that the database process was terminated by the operating system because of excessive memory usage. The application failures occurred immediately after the database service failed.

Did the AI miss anything?

The AI does not identify the original cause of the memory spike. The logs show that memory usage became critical, but they do not reveal which process originally consumed the memory. Therefore, the root cause can only be traced to memory exhaustion, not to the reason behind it.

Did the AI detect something that might have been easy to miss?

Yes. By correlating timestamps between application.log and system.log, it becomes clear that the database failure happened at exactly the same time as the application errors. This connection may be overlooked when examining only one log file.

When would you not trust AI in debugging?

I would not trust AI without verification when logs are incomplete, when critical information is missing, when systems are highly complex, or when security-related incidents are being investigated. AI can identify patterns, but administrators should always verify conclusions using the original log data.

For Task 3, these logs provide a good example of inconsistent logging: the application reports "Database might be slow" even though the database had actually been killed. You can use that observation as an example of a less useful log message because it understates the severity of the problem.